Конструирование программного обеспечения

Домашнее задание №2

DDD · архитектура · паттерны GoF

ПОС



Преамбула

Каждый семестр в Национальном исследовательском университете «Высшая школа экономики» студенты честно (и иногда слишком честно) заполняют СОП. Звёздочки ставятся, комментарии пишутся, дашборды обновляются. Симметрия, однако, хромала: оценка шла в одну сторону, как одностороннее движение у корпуса на Покре.

На одном из совещаний прозвучала фраза, которую нельзя было не превратить в pet-project: «А что, если сделать СОП наоборот?» Маркетинг мгновенно предложил название «ПОС» — Преподавательская Оценка Студентов. Юристы уточнили, что это «не месть, а развитие культуры feedback». Студсовет уточнил, что «мы всё видим». ФКН уточнил, что без доменной модели это будет Excel с обидой.

Ответственным за «техническую часть» по традиции назначили студентов курса по конструированию ПО. Сначала хотели доверить AI агентам, но в итоге договорились о компромиссе, достойном большой инженерии: консольный модуль оценивания + Domain-Driven Design, слоёная архитектура (Ports & Adapters / Onion / Clean) и паттерны GoF.

«Обратная связь – спорт для двоих»,
— сказал лектор по КПО и открыл эту домашку.

Заказчик — учебный офис (на самом деле — семинарист, у которого наконец-то свой СОП наоборот). Подрядчик — вы. Бюджет — две недели.

> *Со слов заказчика: «Нам не нужен ещё один Google Form. Нужна доменная модель: студенты, критерии, оценки, честный итоговый балл по весам, аналитика и выгрузки. Нельзя поставить 12 баллов из 10. Нельзя оценить критерий, которого нет в рубрикаторе. Домен не должен знать про CSV и консоль — сделайте нормальные порты. DDD как на семинаре, архитектура Ports and Adapters (можно и Onion / Clean), паттерны GoF по делу (важно не количество, а именно уместность) »*

Перед вами задача: разработать доменную модель и консольное приложение модуля «ПОС» с применением DDD, одной из слоёных архитектур и ряда паттернов GoF.

Предметная область

Модуль «ПОС» — учёт преподавательской оценки студентов: кто учится на курсе, по каким критериям оценивают и какие оценки уже выставлены. В будущем можно нарастить пересдачи, апелляции и несколько курсов — сейчас достаточно ядра.

Язык предметной области (ubiquitous language):

- ***студент*** — кого оценивают;
- ***критерий*** — за что ставят балл, у критерия есть максимум и вклад в итог;
- ***оценка*** — факт «этому студенту по этому критерию поставили такой балл» (дата и комментарий — по желанию);
- ***итог*** — результат по студенту, полученный из его оценок и рубрикатора, а не «отдельная цифра наугад».

Правила, которые модель не должна позволять нарушить:

- балл не выходит за пределы критерия;
- оценку можно выставить только по критерию из рубрикатора;
- итог согласован с текущим набором оценок (считается в Domain Service).

Это словарь для разговора с «заказчиком» и для кода. Какие будут entity, value object и границы агрегатов — решаете вы и кратко объясняете в README; готовой классовой схемы в ТЗ нет.

Ограничения и рекомендации: DDD

Минимальный словарь DDD (покажите в коде и README):

- Ubiquitous Language — те же слова, что в предметной области;
- Entity и Value Object — осознанное разделение (с примером в README);
- инварианты — правила из предметной области охраняются моделью;
- Repository (порт) — хранение снаружи домена;
- Domain Service — если правило не принадлежит одной сущности (типичный кандидат — расчёт итога).

Анемичные DTO + «сервисы-проходники» без инвариантов как доменная модель не засчитываются.

Ограничения и рекомендации: архитектура

Нужно реализовать одну из архитектур на выбор: Ports and Adapters (Hexagonal), Onion Architecture или Clean Architecture. Достаточно учебной, «облегчённой» версии — без микросервисов.

Подсказки (общая идея у всех трёх похожа):

- В центре — Domain: entities, VO, инварианты, domain services. Без ссылок на UI, файлы, фреймворки.

- Снаружи домена — Application / Use cases: сценарии «выставить оценку», «посчитать итог», «импортировать ведомость».
- Ещё снаружи — Infrastructure / Adapters: JSON/CSV-хранилище, консольный UI, маппинг DTO ↔ домен.
- Зависимости направлены внутрь: инфраструктура зависит от абстракций (портов), а не наоборот. DI собирает граф на краю (composition root).

В README укажите выбранную архитектуру, слои/папки и стрелку зависимостей.

Ограничения и рекомендации: паттерны GoF

Не нужно коллекционировать паттерны. Достаточно уместно применить по одному (максимум два) паттерну из каждой группы:

- порождающие — единая точка создания с инвариантами;
- структурные — не размазывать сценарии/модификаторы по UI;
- поведенческие — сменяемое правило или общий каркас с вариациями.

Паттерн засчитывается, если закрывает боль домена или приложения, а не существует «для галочки». Идеи можно взять с семинаров или предложить свои. Обязательно опишите в README: какой паттерн, где, какую боль закрывает. Ниже примеры, которые можно не реализовывать, а предложить свои:

Порождающие

- Фабрика — создание Evaluation с проверкой score ∈ [0, max_score] и связи с рубрикатором
- Builder — сборка рубрикатора

Структурные

- Decorator — округление поверх базового Score + множество иных особенностей подсчета.
- Facade — единая точка сценариев для консоли.

Поведенческие

- Strategy — разные правила итога в Domain Service.
- Template Method — импорт ведомостей: общий каркас, разный парсинг CSV/JSON.

Паттерн засчитывается только если решает задачу. Три уместных паттерна из разных групп ценнее шести декоративных.

Требуется

1. Консольное приложение модуля «ПОС» с основными сценариями: студенты, критерии, оценки; расчёт итога; понятный ввод/вывод.
2. Доменная модель в духе DDD (ubiquitous language, entity/VO, инварианты, repository, domain service).
3. Одна из архитектур: Ports and Adapters, Onion или Clean Architecture — с явными слоями и зависимостями внутрь.
4. Паттерны GoF: минимум по одному из каждой группы (порождающие / структурные / поведенческие), максимум два в группе.
5. README: логика решений, карта DDD, архитектура и слои, обоснование GoF, роль ИИ в вашем решении.
6. DI-контейнер; желательны несколько юнит-тестов на инварианты и/или Domain Service.

Не переигрывайте: за «мёртвую» архитектуру из пустых папок и overengineering оценку снизим.

Функциональные требования

1. **Студенты и рубрикатор.** Можно добавить студента и критерии оценивания (название, `max_score`, вес), а также посмотреть текущий список (изменение и удаление — по необходимости, не самоцель)
2. **Выставление оценок.** Можно выставить оценку студенту по критерию, посмотреть оценки и при необходимости исправить или отменить. Главное — чтобы нельзя было нарушить правила домена (см. п. 3).
3. **Инварианты и итог.**
    - `score` в диапазоне критерия;
    - оценка только по существующему критерию рубрикатора;
4. **Демонстрация.** В консоли прогоните сквозной сценарий: завести студентов и критерии → выставить оценки → получить итог → экспорт итога.

Политика использования ИИ

ИИ-помощники разрешены как инструмент. Студент отвечает за понимание и качество кода. Если претендуете на 8, 9 и 10 баллов, на короткой защите нужно объяснить решение и уметь доработать его под новое требование. Можно отказаться от защиты — тогда оценки 8, 9 и 10 снижаются до 7 баллов.

Критерии оценки (10 баллов)

№	Критерий	Баллы
1	Функциональные требования (CRUD) и работающее консольное приложение	2.5
2	DDD: UL, entity/VO, инварианты, repository, domain service	2
3	Архитектура: Ports and Adapters / Onion / Clean (слои + зависимости внутрь)	2
4	GoF: ≥1 паттерн из каждой группы (можно до 2); уместность важнее количества	2.5
5	DI-контейнер (+ тесты инвариантов желательны); README	1

Штрафы и защита

- до −2 баллов за ошибки во время выполнения;
- до −5 баллов, если программа не собирается;
- снижение пропорционально невыполненным функциональным требованиям;
- до −1.5 балла за «паттерн/архитектуру ради галочки» или анемичную модель без инвариантов;
- −1 балл за грубое игнорирование кодстайла.